OPENFOUNDRY · PRODUCT AUDIT

Dummy Features Audit

Screens from the running app with the simulated, unenforced or cloud-only areas marked in red.

DATE
3 November 2026

BUILD
Local workspace · v0.2.0

SCOPE
6 findings across Manage, Build and Operate

METHOD
Live UI capture at 1440 px · elements located in the DOM

> The core is real. Chat, deployments, uploaded-file RAG, batch jobs, evaluations, routines, workflows, the AI gateway, MCP/HTTP tools, traces and benchmarks all have working local backends. The findings below are the places where the UI still claims a cloud feature that does not exist, or fails to enforce what it displays.

Summary

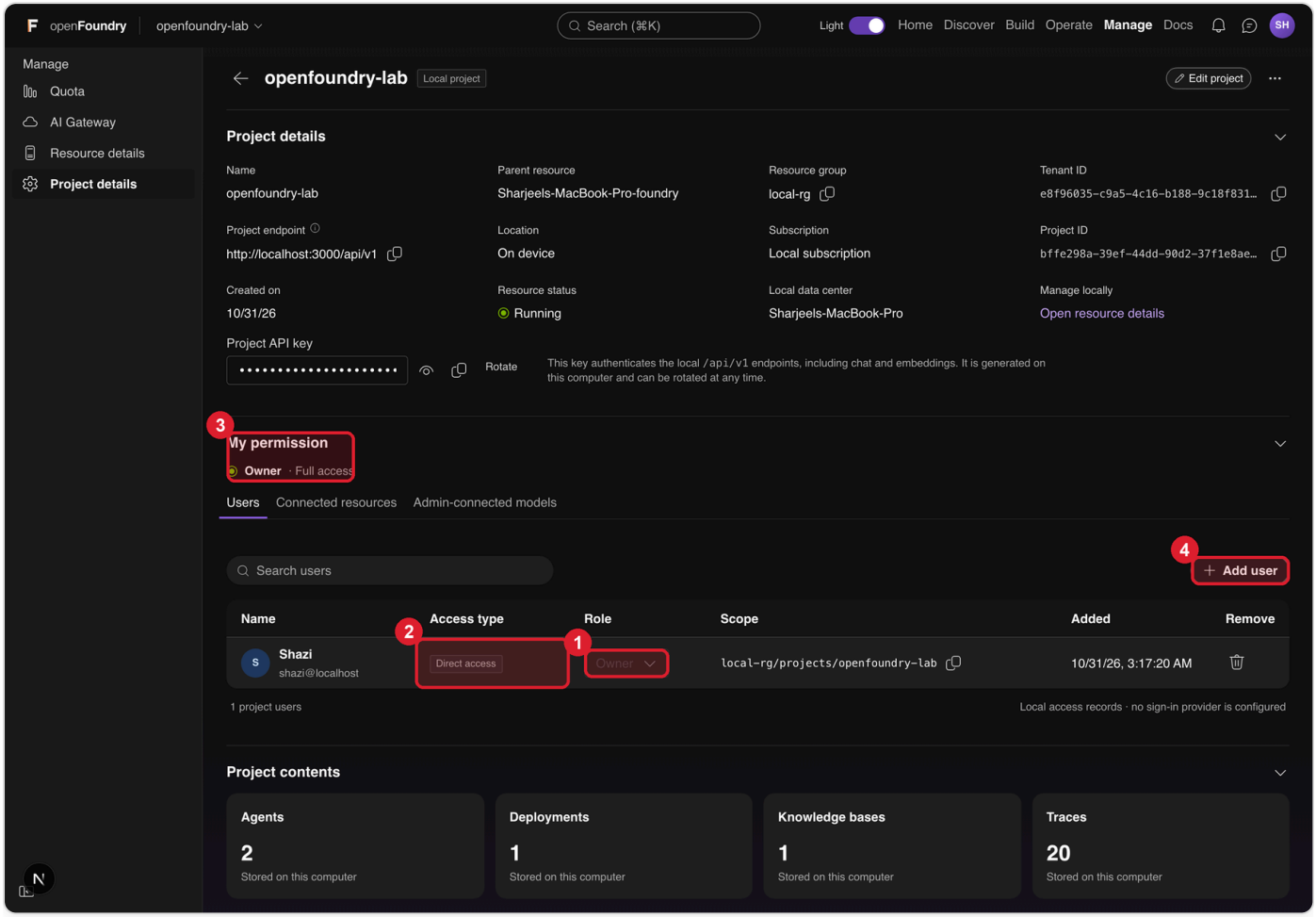
Six features are marked. Two are actively misleading because they imply data access or security that is not enforced; three are dead-end cloud chrome; one is cosmetic ceremony over real data.

#	FEATURE	SEVERITY	WHY IT IS DUMMY
1	User roles & access	High	Roles and access types exist, but no sign-in or authorization enforces them.
2	Knowledge sources & connection	High	Non-upload sources show “Connected” but never ingest; the only connection is seeded.
3	Quota → Request increase	Medium	Writes a request to JSON that nothing can approve or apply.
4	Agent status & Stop	Medium	Status is a label; a “Stopped” agent’s endpoint keeps answering.
5	Subscription / project identity	Low	Random UUIDs and constant names around one local machine; one-option pickers.
6	Compliance → Re-assess	Low	“Re-assess” is a page reload; “Last assessed” is always now.

1 User roles & access

High

Manage → Project details → Users



The user list is real data; the permissions it displays are not.

1. Role dropdown — Owner / Contributor / Reader.
2. Access type — “Direct access” / “Inherited access”.
3. “My permission · Owner · Full access”.
4. Add user — creates another access record.

WHAT IS DUMMY

The UI models an access-control system, but nothing enforces it. There is no sign-in, and no management API checks a role; `/api/users` and `/api/settings` are unauthenticated on localhost. Runtime endpoints accept a single project key or resource key (`src/lib/auth.ts`). Changing “Owner” to “Reader” changes a string and nothing else. The footer even says “Local access records · no sign-in provider is configured”.

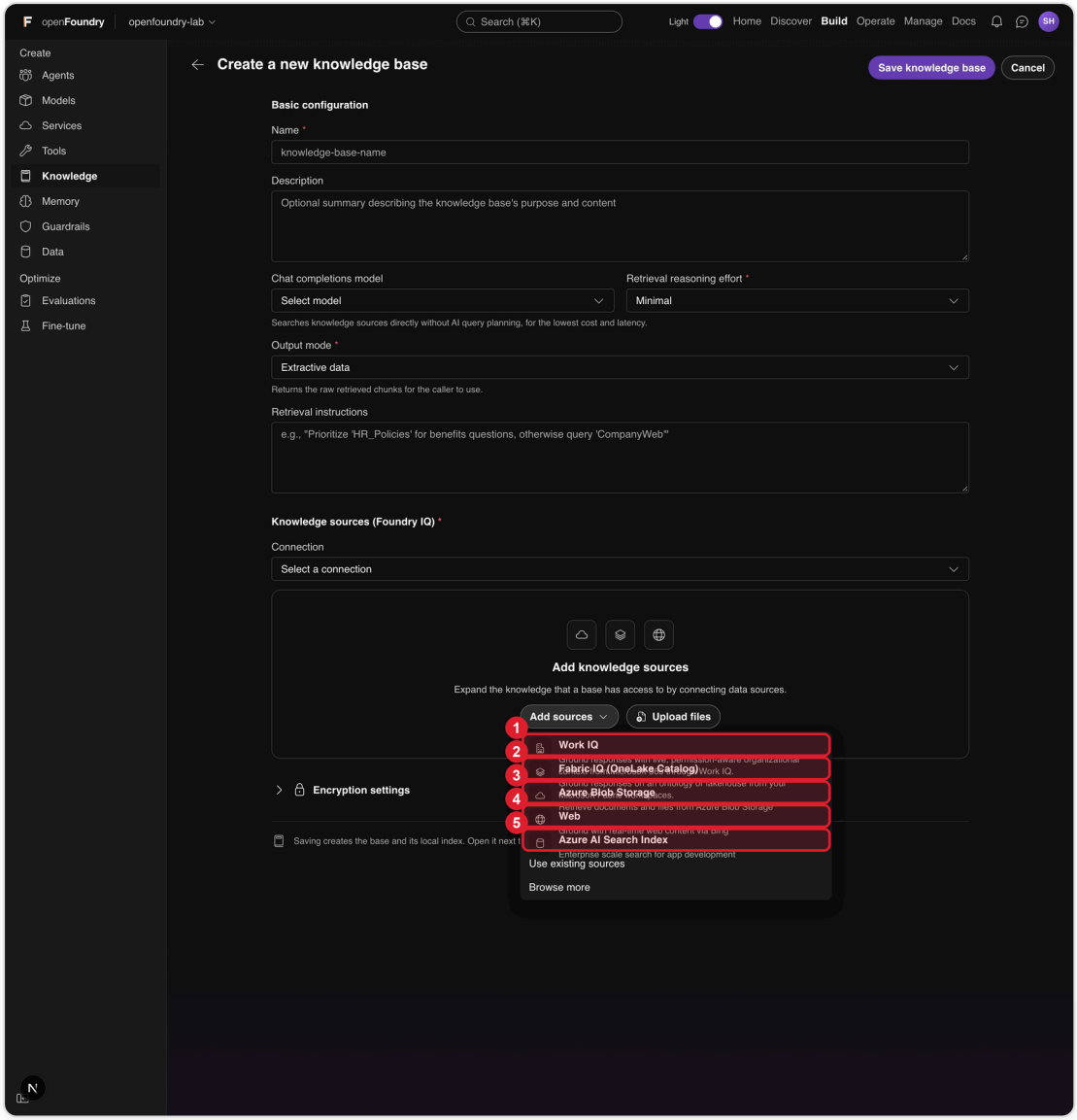
FIX

Implement real local auth (a per-user passcode or session) and enforce role checks on write endpoints — or relabel this as “Collaborators (local notes)” and remove the role and access-type controls.

2 Knowledge sources that never ingest

High

Build → Knowledge → Create



Five source types, none of which fetches or indexes content.

1. Work IQ — “live, permission-aware organizational context”.
2. Fabric IQ — ontology or lakehouse from Fabric.
3. Azure Blob Storage — documents from a container.
4. Web — “real-time web content via Bing”.
5. Azure AI Search Index — enterprise search index.

WHAT IS DUMMY

All five sources can be added and immediately show “Connected”. Only uploaded files are actually ingested and indexed. `addSource` records the kind with `status: "Connected"` and `documentCount: 0`; retrieval (`src/lib/rag.ts`) only searches uploaded documents. A knowledge base can therefore look connected and still answer from nothing.

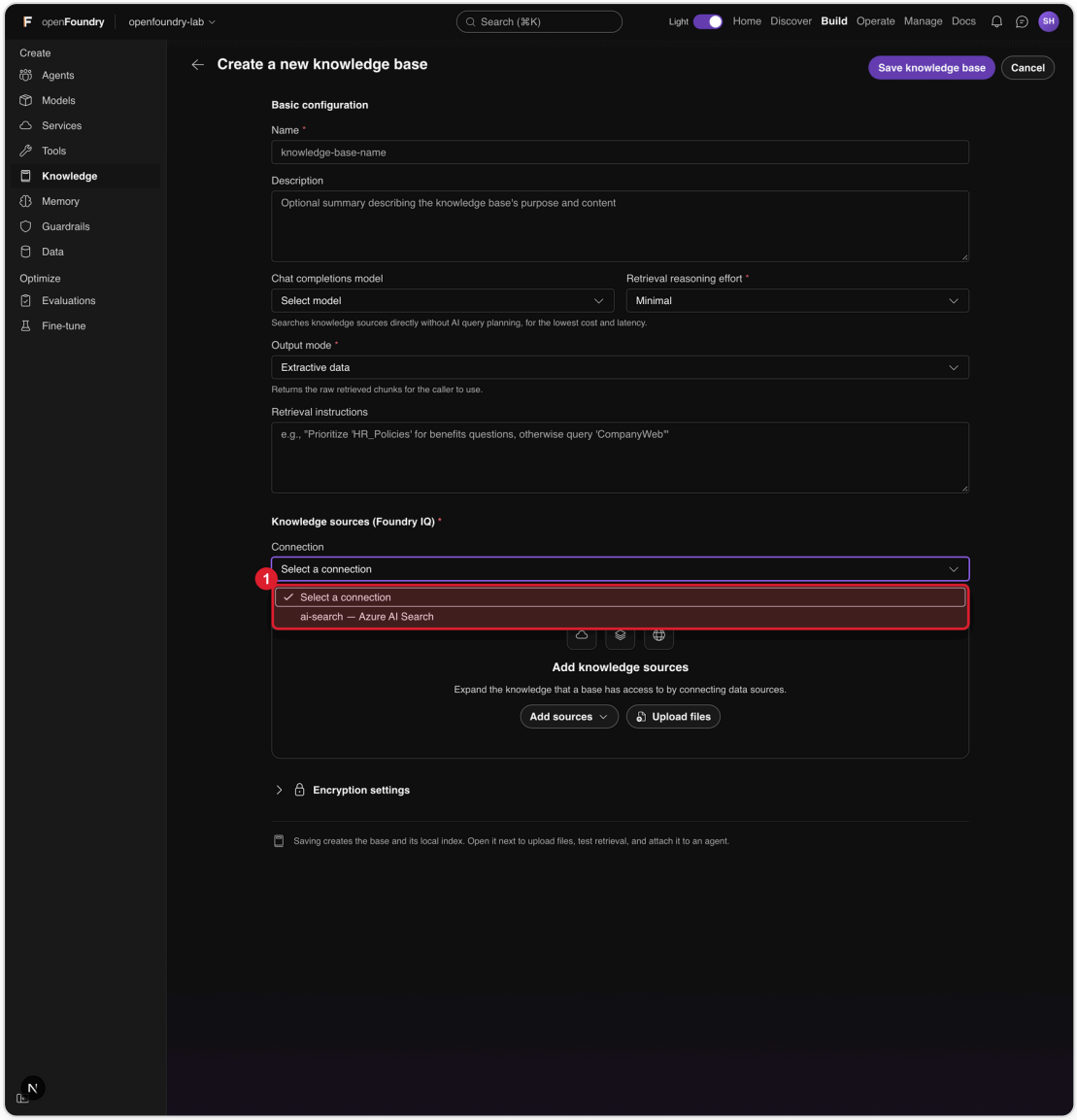
FIX

Show “Connected” only after ingestion succeeds. Label non-ingesting kinds “Connection only — not indexed”, or hide them until implemented. Never let an empty base look healthy.

2 The only connection is seeded

High

Build → Knowledge → Connection



A seeded connection with no creation, test or auth path.

1 "ai-search — Azure AI Search", pointing at https://localhost.search.local.

WHAT IS DUMMY

The Connection picker offers a real-looking Azure AI Search connection on first run (`src/lib/store.ts:275`). The endpoint `https://localhost.search.local` does not resolve, and `/api/connections` only exposes GET — there is no way to create, test or authenticate a real connection. Selecting it scopes a knowledge base to an endpoint that does not exist.

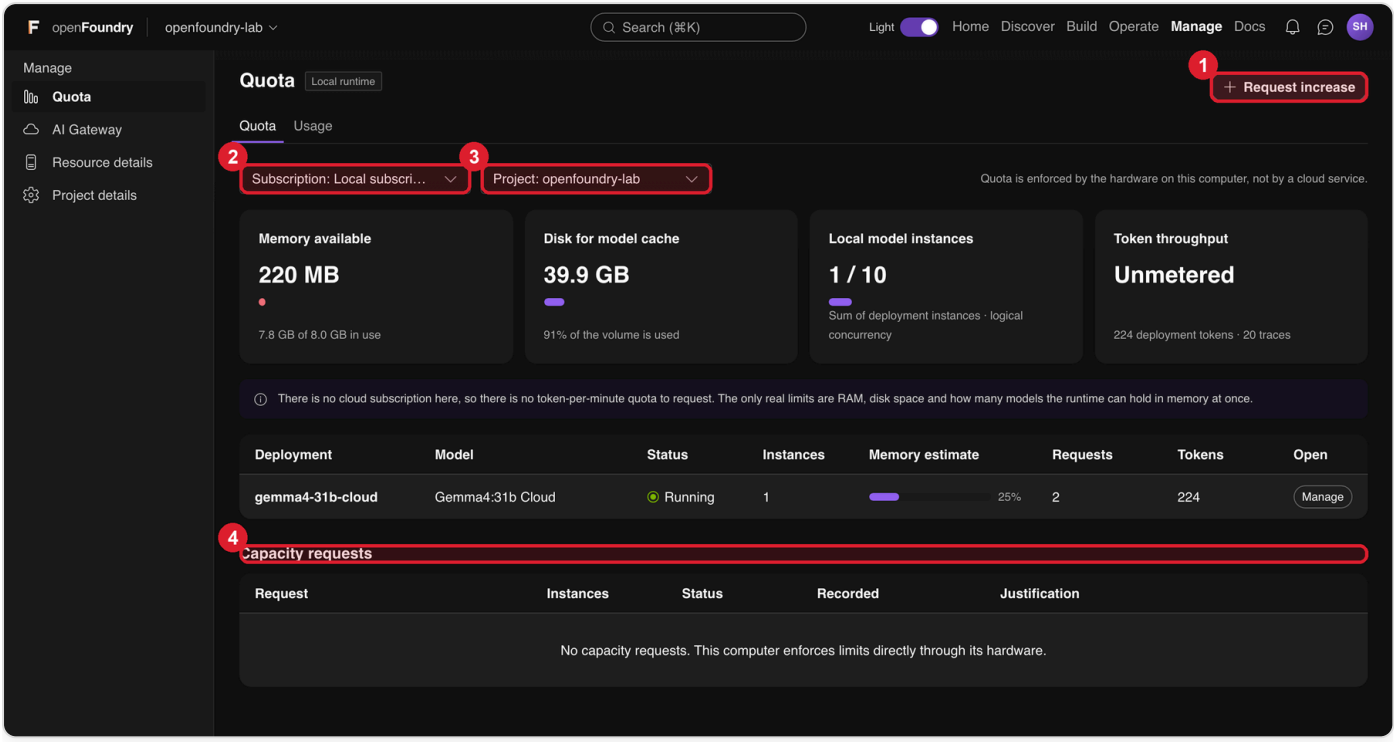
FIX

Remove the seeded connection, or mark it clearly as a placeholder and require a real endpoint plus a successful test before it can be selected.

3 Quota → Request increase

Medium

Manage → Quota



A request/approval workflow with no approver.

1. Request increase — opens a form that records a request.
2. Subscription picker — exactly one option.
3. Project picker — exactly one option.
4. Capacity requests — the table the request lands in.

WHAT IS DUMMY

The page is honest that hardware enforces limits, yet it offers an Azure-style “Request increase” flow. The button writes to `quota-requests.json` (`/api/quota-requests`); nothing can approve or apply the request, and the table underneath simply lists what you wrote. The Subscription and Project pickers each contain exactly one value.

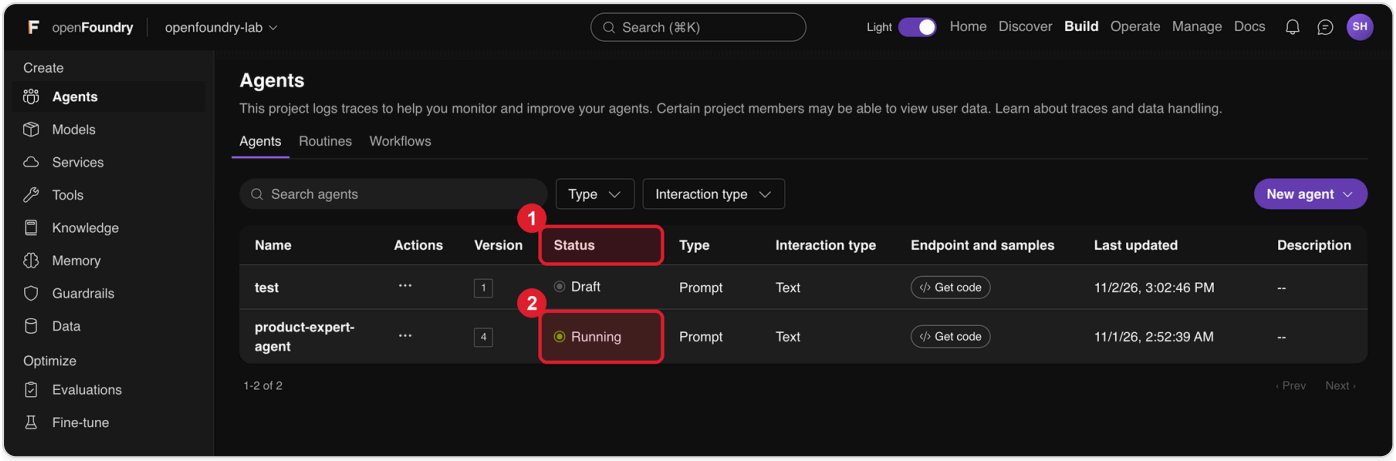
FIX

Remove the request flow, or turn it into “Free memory” guidance that links to the deployments holding RAM. Drop the single-option pickers.

4 Agent status and Stop

Medium

Build → Agents · and the agent builder



The status column is real data, but it gates nothing.

1. Status column — Draft / Running, presented as a lifecycle.
2. The same column for a second agent: the badge is a stored string.

WHAT IS DUMMY

Publish and Stop only assign `agent.status` (`src/app/api/agents/[id]/route.ts`). The per-agent endpoint `/api/v1/agents/[id]/chat/completions` never checks that status, so a “Stopped” agent keeps answering requests. The lifecycle looks like a deployment, but it is a label; nothing is unloaded or disabled.

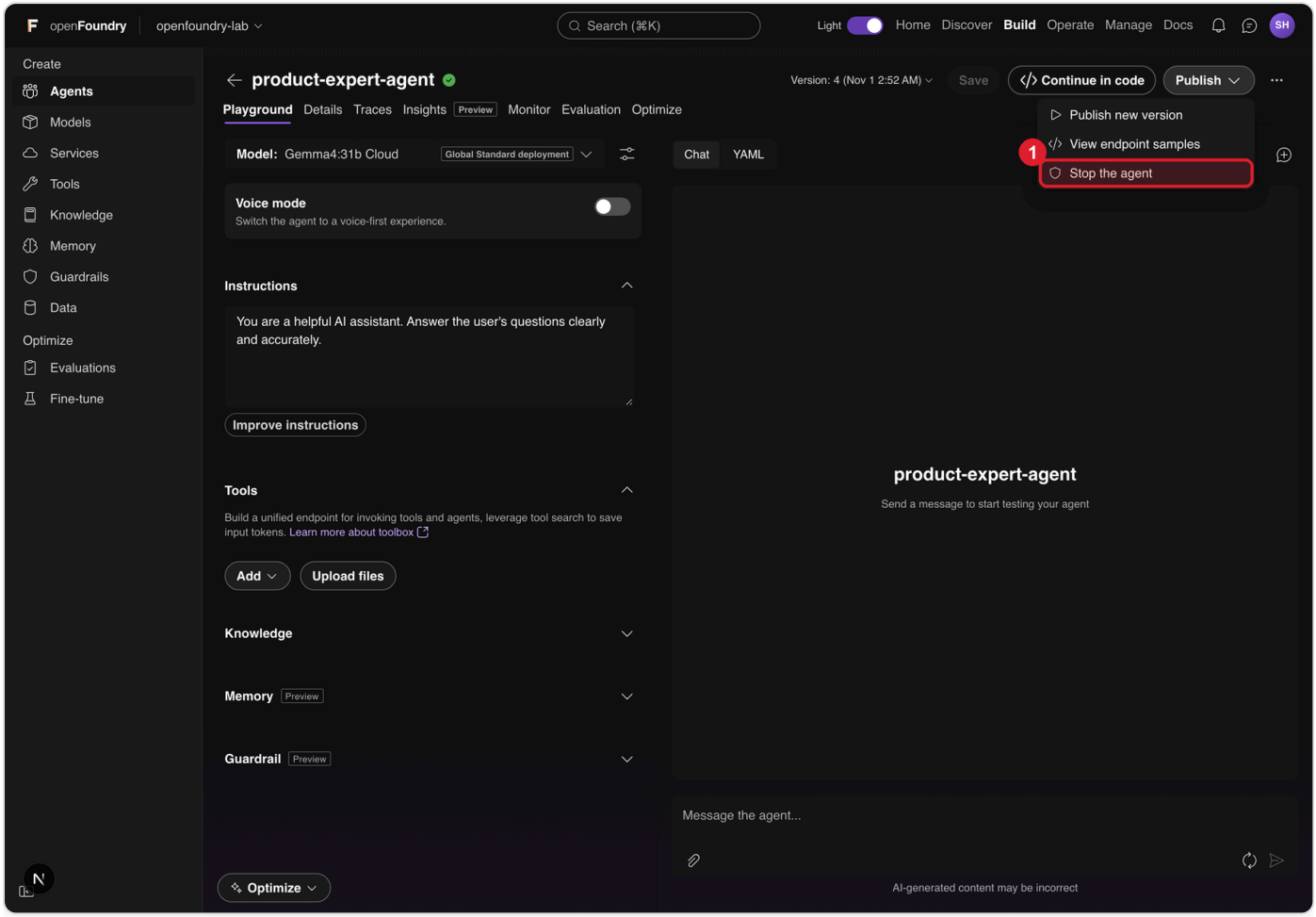
FIX

Gate the endpoint on status and actually unload or disable the agent on stop — or rename the states to Draft / Published and remove “Stop”.

4 “Stop the agent” does not stop it

Medium

Build → Agent builder → Publish menu



The same label seen from the builder.

1 Stop the agent — sets status to “Stopped”; the endpoint keeps serving.

WHAT IS DUMMY

The builder presents “Stop the agent” next to “Publish new version”, implying a lifecycle. The action only writes a status string. Requests sent to the agent’s endpoint still reach the model, because the endpoint never reads that status.

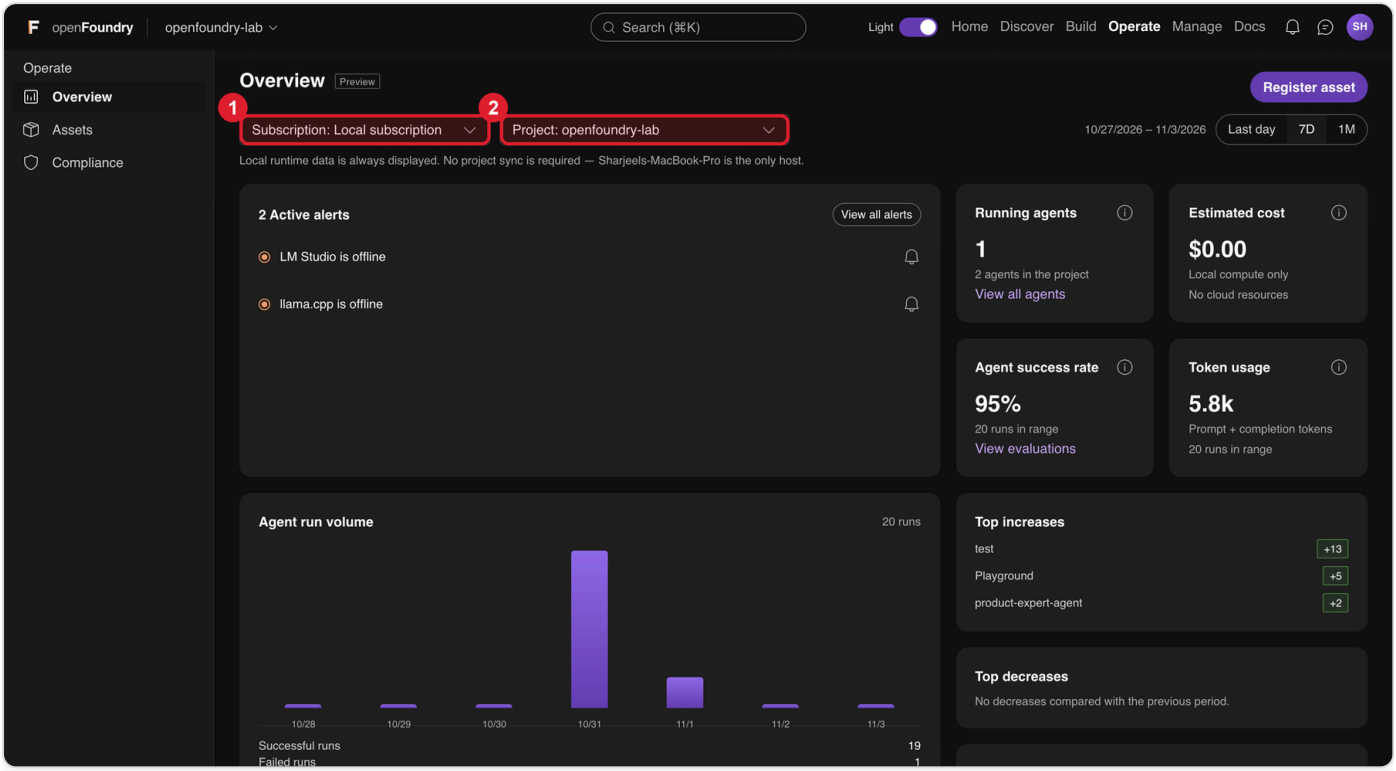
FIX

Either make Stop revoke or pause the agent’s key and reject calls, or remove the action and call the states Draft / Published.

5 Subscription, project and resource identity

Low

Operate → Overview



A cloud hierarchy with exactly one item at every level.

1. Subscription picker — “Local subscription”; one option.
2. Project picker — the one local project; one option.

WHAT IS DUMMY

These are Azure’s resource-hierarchy nouns wrapped around one local machine. `subscriptionId` is a random UUID generated on first run (`src/lib/store.ts:519`); the names are constants (“Local subscription”, “local-rg”, “<hostname>-foundry”). There is no tenant, subscription, resource group or project isolation — one data directory, one key. The one-option dropdowns are the tell.

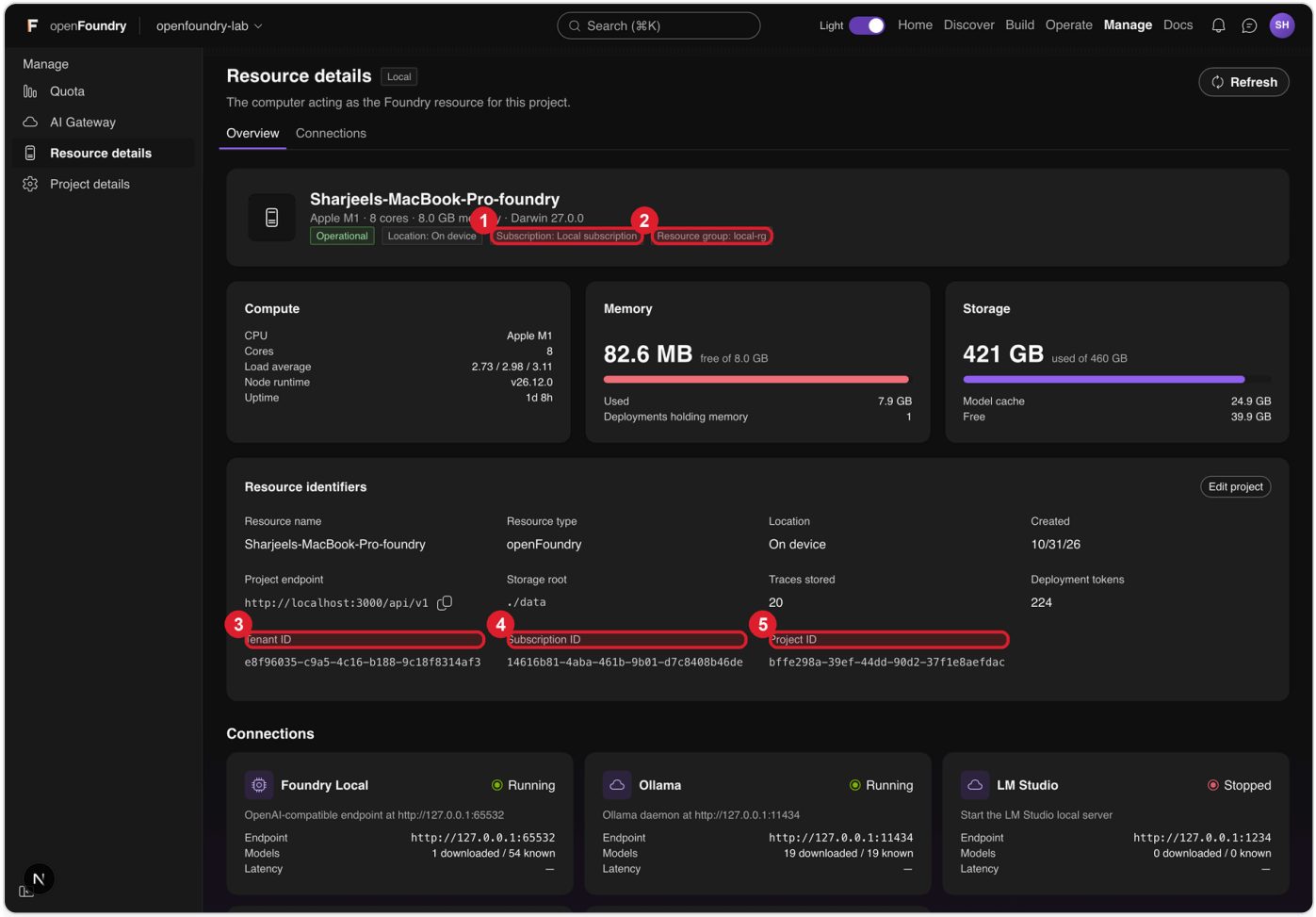
FIX

Replace the hierarchy with one “Workspace” identity (name, path, key). If IDs are needed for export, label them “export identifiers” and keep them out of the everyday UI.

5 Random IDs presented as cloud identity

Low

Manage → Resource details



Three identifiers, zero cloud resources behind them.

1. Subscription chip — constant label.
2. Resource group chip — constant “local-rg”.
3. Tenant ID — random UUID.
4. Subscription ID — random UUID.
5. Project ID — random UUID.

WHAT IS DUMMY

The resource page presents Tenant ID, Subscription ID and Project ID as if they identified cloud resources, but they are random UUIDs minted locally. Nothing else in the app resolves or validates them. They exist only to make the page look like an Azure resource blade.

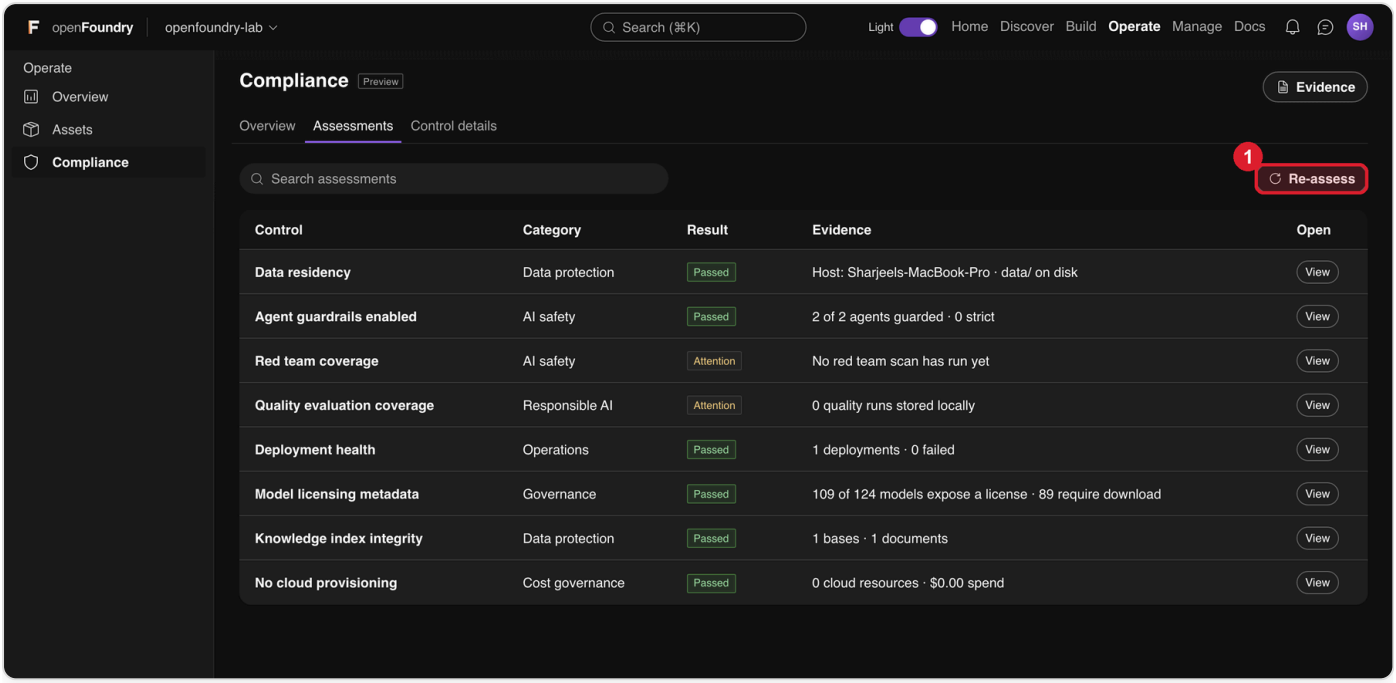
FIX

Remove the IDs, or label the block “Export identifiers” and generate them only when the user exports a deployment to a cloud.

6 Compliance → Re-assess

Low

Operate → Compliance → Assessments



The results are real; the assessment act is a refresh.

1 Re-assess — reloads the page.

WHAT IS DUMMY

The controls are derived from real state, so the posture itself is real. The assessment ceremony is not: “Re-assess” is `window.location.reload()` (`src/app/operate/compliance/page.tsx:327`), and “Last assessed” is always the current time. Nothing is stored, compared or versioned.

FIX

Store a real assessment run with a timestamp and a diff against the previous run — or remove the button and the timestamp.

What is *not* dummy

It is worth being explicit, because the list above is short compared with the surface area of the product:

> **Real local backends:** chat and streaming, deployments and per-deployment keys, uploaded-file RAG with citations, batch jobs, evaluations (heuristic scoring, but real model runs), routines, workflows, the AI gateway with routes and caching, MCP and HTTP tools, traces, and benchmarks measured from traces.

The pattern in every finding is the same: **a cloud noun was kept after the cloud was removed.** “User role”, “Connected source”, “capacity request”, “tenant”, “subscription” and “re-assess” all describe Azure processes that have no local equivalent. Rename each one to its local reality — collaborator, indexed source, memory pressure, workspace — and the product stops reading as a Foundry skin.

Captured from the running workspace on 3 November 2026. Red boxes mark the exact elements; coordinates were taken from the DOM at capture time.